

YZM0206 Laboratuvar 7 Raporu

Ad Soyad: Yusuf Yıldız **Öğrenci No:** 24458667016

1. Amaç ve Hedefler Bu laboratuvarın amacı, K-En Yakın Komşuluk (K-Nearest Neighbors - KNN) algoritmasını kullanarak bir sınıflandırma modeli geliştirmek ve K hiperparametresinin modelin başarımına olan etkisini incelemektir. Bu doğrultuda bir telekomünikasyon şirketine ait müşteri veri seti kullanılarak müşterilerin aldıkları hizmet kategorisi tahmin edilmiştir.

2. Veri Setinin İncelemesi ve Sınıf Dağılımı Veriler pandas kütüphanesi ile yüklenerek bağımlı hedef değişken olan `custcat` kolonunun dağılımı incelenmiştir. Aşağıdaki kod ve çıktıda görüleceği üzere sınıflara ait müşteri sayıları; 281, 266, 236 ve 217 olarak tespit edilmiştir.

```
... --- Veri Setinin İlk 5 Satırını ---  
   region tenure age marital address income ed employ retire gender reside custcat  
0      2     13  44      1      9    64.0  4      5     0.0      0      2      1  
1      3     11  33      1      7   136.0  5      5     0.0      0      6      4  
2      3     68  52      1     24   116.0  1     29     0.0      1      2      3  
3      2     33  33      0     12    33.0  2      0     0.0      1      1      1  
4      2     23  30      1      9    30.0  1      2     0.0      0      4      3  
  
--- Sınıf Dağılımı (Müşteri Kategorileri) ---  
custcat  
3      281  
1      266  
4      236  
2      217  
Name: count, dtype: int64
```

Teknik Değerlendirme: Etiketler (sınıflar) matematiksel olarak kusursuz bir üniform dağılıma sahip olmasalar da, sınıf sayıları birbirine oldukça yakındır. Makine öğrenmesi perspektifinden bu veriler "dengeli (balanced)" olarak kabul edilir. Bu denge sayesinde model eğitilirken çoğunluk sınıfının tahmini domine etmesi engellenmiş olup, veri seti KNN algoritmasının eğitimine doğrudan uygundur.

3. Veri Hazırlığı ve Normalizasyon Scikit-learn ile çalışabilmek adına hedef kolon veri setinden ayrılmış ve ardından bağımsız değişkenler (X) standardize edilmiştir.

```

X = df.drop('custcat', axis=1).values
y = df['custcat'].values

scaler = StandardScaler()
X_normalized = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_normalized, y, test_size=0.2, random_state=42)

print("Eğitim Seti (X_train):", X_train.shape)
print("Test Seti (X_test):", X_test.shape)

Eğitim Seti (X_train): (800, 11)
Test Seti (X_test): (200, 11)

```

Teknik Gerekçe: KNN algoritması, noktalar arasındaki uzaklığı (Öklid, Manhattan vb.) ölçerek çalışır. Veri setindeki kolonlar arasında birim farklılıkları (örneğin gelirin on binlerce, yaşı onlarca değerde olması) olduğu için, büyük değerli özelliklerin algoritmada diğerlerini ezmesini önlemek adına StandardScaler ile tüm veriler aynı ölçeğe (normalize) getirilmiştir.

4. Modellerin Eğitimi (K=4 ve K=6 Karşılaştırması) Makine öğrenmesi prensipleri gereğince veri seti %80 Eğitim ve %20 Test olarak ayrılmıştır. İlk etapta komşuluk sayısı (K) 4 ve 6 seçilerek modeller test edilmiştir.

```

... --- K=4 İçin Performans Sonuçları ---
Accuracy Score: 0.3250
          precision    recall  f1-score   support

         1         0.37         0.48         0.42         60
         2         0.15         0.13         0.14         39
         3         0.32         0.31         0.31         55
         4         0.40         0.30         0.35         46

    accuracy                    0.33         200
  macro avg         0.31         0.31         0.30         200
 weighted avg         0.32         0.33         0.32         200

--- K=6 İçin Performans Sonuçları ---
Accuracy Score: 0.3300
          precision    recall  f1-score   support

         1         0.35         0.52         0.42         60
         2         0.26         0.23         0.24         39
         3         0.35         0.31         0.33         55
         4         0.32         0.20         0.24         46

    accuracy                    0.33         200
  macro avg         0.32         0.31         0.31         200
 weighted avg         0.33         0.33         0.32         200

```

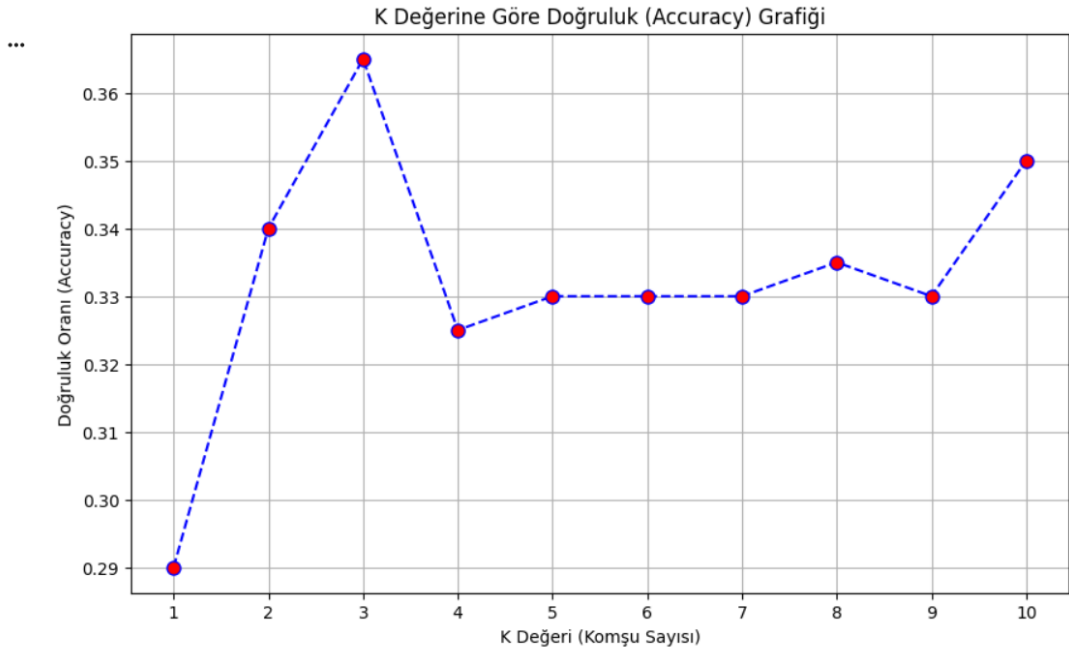
Yorum: K=4 yerine K=6 komşuya bakıldığında test seti üzerindeki doğruluğun (Accuracy) ufak bir miktar arttığı (%32.5'ten %33.0'a) gözlemlenmiştir. Ancak bu ufak deneme, K değerini manuel olarak seçmek yerine tüm olasılıkların denendiği bir sistem kurulması gerektiğine işaret etmiştir.

5. Optimum K Değerinin Bulunması Doğru ve optimum K değerini tespit edebilmek için algoritma bir döngü içerisine alınarak K'nın 1'den 10'a kadar olan değerleri için baştan eğitilmiş ve doğruluk oranları test edilmiştir.

...

--- 1'den 10'a Kadar K Değerleri İçin Doğruluk Skorları ---

K=1 -> Accuracy: 0.2900
K=2 -> Accuracy: 0.3400
K=3 -> Accuracy: 0.3650
K=4 -> Accuracy: 0.3250
K=5 -> Accuracy: 0.3300
K=6 -> Accuracy: 0.3300
K=7 -> Accuracy: 0.3300
K=8 -> Accuracy: 0.3350
K=9 -> Accuracy: 0.3300
K=10 -> Accuracy: 0.3500



- **K=1** kullanıldığında model test veri setinde en düşük skoru (%29) vermiştir. Bunun teknik sebebi, komşuluk 1'e düştüğünde modelin çok fazla detaya inerek gürültüleri (noise) ezberlemesi ve **aşırı öğrenme (Overfitting)** problemi yaşamasıdır.
- **K değeri büyütüldükçe** (örn. 5, 6, 7 vb.) komşuluk çemberi çok fazla genişlediği için model gürültüleri filtrelese de aşırı genelleştirme (Over-generalization) eğilimi göstermiş ve doğruluk oranı optimum seviyenin altında kalmıştır. Analiz

doğrultusunda, bu sınıflandırma problemi için gürültü ile genelleştirme arasındaki en iyi dengenin $K=3$ noktasında kurulduğu tespit edilmiştir.